

Git for Developers

Module 1 — Introduction & Basics

Why Git? Installation, Setup, and Your First Repository

Module 1 — Agenda

Version Control Fundamentals

- Why version control? The problems Git solves
- Centralized vs. distributed version control systems
- What is Git? Snapshots, commits, and history

Getting Started

- Installing Git on macOS, Linux, and Windows
- First-time setup: identity, editor, defaults
- Git Basics: local vs. remote repositories

Part 1 — Why Version Control?

Life Without Version Control

Traditional approach to managing code:

- Saving copies: `project_final.zip`, `project_final_v2.zip`,
`project_FINAL_REAL.zip`
- Emailing files back and forth between team members
- Manually tracking who changed what and when
- Overwriting a colleague's changes by accident
- No way to roll back a broken change

"It worked yesterday!" — every developer without version control

What Version Control Solves

Problem	Git solution
"Who changed this?"	Every commit is attributed to an author
"What changed?"	Full diff history of every file
"When did this break?"	<code>git bisect</code> finds the exact commit
"Can I undo this?"	Revert or reset to any previous state
"How do we work in parallel?"	Branches let everyone work independently
"Which version is in production?"	Tags and releases mark specific commits

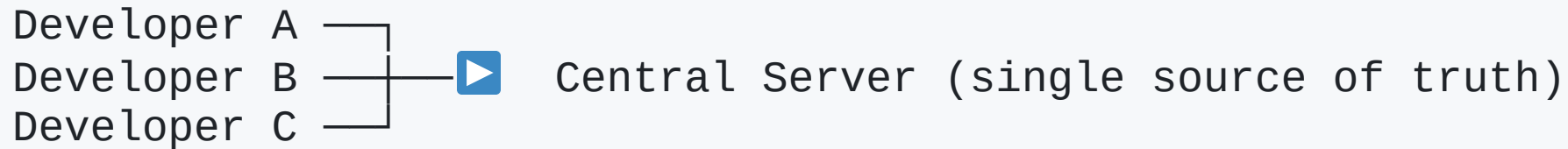
What Version Control Enables

- **Collaboration:** multiple people work on the same codebase simultaneously
- **Experimentation:** try ideas in branches — discard without consequence
- **Audit trail:** every change is recorded with author, timestamp, and message
- **Deployment confidence:** deploy known, tested, tagged commits
- **Code review:** changes are reviewed before they reach the main branch

Part 2 — Centralized vs. Distributed VCS

Centralized Version Control (CVCS)

Examples: **SVN** (Subversion), **CVS**, **Perforce**



How it works

- All history lives on a central server
- Developers check out a **working copy** — no local history
- Every commit, diff, and branch operation **requires a server connection**
- If the server goes down, nobody can commit

Distributed Version Control (DVCS)

Examples: **Git**, **Mercurial**



How it works

- Every developer has a **complete copy** of the repository including all history
- Commits, diffs, branches, and logs work **fully offline**
- The remote server is just another repository — by convention it's the shared origin
- Losing the server does not lose history — any clone can restore it

CVCS vs. DVCS Comparison

Feature	Centralized (SVN)	Distributed (Git)
History storage	Server only	Every clone
Works offline	No	Yes
Speed	Network-dependent	Mostly local
Branching	Heavy, slow	Lightweight, instant
Backup	Single point of failure	Every clone is a backup
Learning curve	Lower	Steeper initially

Git's distributed model is the reason branches are cheap and fast.

Part 3 — What is Git?

What is Git?

- A **distributed version control system** created by **Linus Torvalds** in 2005
- Originally built to manage the Linux kernel source code
- Tracks changes to files as a series of **snapshots**, not diffs
- Designed for **speed**, **integrity**, and **non-linear development**
- The most widely used version control system in the world

"Git is not GitHub." Git is the tool; GitHub, GitLab, and Bitbucket are hosting platforms built on top of Git.

How Git Thinks About Data

Other VCSes — Delta-based storage

```
Version 1:  A _____  
Version 2:  A ——— Δ1 _____  
Version 3:  A ——— Δ1 ——— Δ2 _____
```

Git — Snapshot-based storage

```
Version 1:  [snapshot of entire project at V1]  
Version 2:  [snapshot of entire project at V2] ← unchanged files are just pointers  
Version 3:  [snapshot of entire project at V3]
```

Git stores a snapshot of **all tracked files** with every commit. Unchanged files are stored as references, not duplicates.

The Three Local Areas

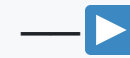
Working Directory

Your files as
you edit them



Staging Area (Index)

Files you've
staged for the
next commit



Local Repository

Committed
snapshots
(permanent history)

Area	What it contains
Working Directory	Your actual files — modified, new, deleted
Staging Area	A preview of what the next commit will look like
Repository	All committed snapshots, stored in <code>.git/</code>

Commits, Trees, and History

A **commit** is a snapshot containing:

- A pointer to the project's file tree at that moment
- Author name and email
- Timestamp
- Commit message
- A pointer to the **parent commit(s)**

A ← B ← C ← D (each arrow = "parent of")
HEAD points to D

The chain of commits forms your **history** — immutable and cryptographically verified by SHA hashes.

Part 4 — Installing Git & First-Time Setup

Installing Git

macOS

```
# Option 1 – Homebrew (recommended)
brew install git

# Option 2 – Xcode Command Line Tools (includes Git)
xcode-select --install
```

Linux (Debian/Ubuntu)

```
sudo apt update && sudo apt install git
```

Windows

```
# Option 1 – Winget  
winget install --id Git.Git -e  
  
# Option 2 – Chocolatey  
choco install git
```

Verify Installation

```
$ git --version  
git version 2.45.2
```

Check your PATH

```
which git      # macOS/Linux – should return /usr/bin/git or /usr/local/bin/git  
where git     # Windows – should return the Git installation path
```

First-Time Setup — Identity

Git attaches your name and email to every commit you make.

```
# Set your name and email (stored globally in ~/.gitconfig)
git config --global user.name "Jane Smith"
git config --global user.email "jane@example.com"
```

Use the same email as your GitHub/GitLab account — it links your commits to your profile.

First-Time Setup — Editor and Defaults

```
# Set your preferred editor for commit messages
git config --global core.editor "code --wait"      # VS Code
git config --global core.editor "vim"              # Vim
git config --global core.editor "nano"             # Nano

# Set the default branch name to 'main' (modern standard)
git config --global init.defaultBranch main

# Enable helpful color output
git config --global color.ui auto
```

Viewing Your Configuration

```
# Show all config values and their source  
git config --list --show-origin
```

```
# Show a specific value  
git config user.name
```

```
# Edit the global config file directly  
git config --global --edit
```

Config file locations

Scope	File	Applies to
<code>--system</code>	<code>/etc/gitconfig</code>	All users on the machine
<code>--global</code>	<code>~/.gitconfig</code>	All repos for your user
<code>--local</code>	<code>.git/config</code>	This repository only

Local config overrides global, which overrides system.

Part 5 — Git Basics: Repositories

Local vs. Remote Repositories

Type	Location	Purpose
Local	Your machine	Where you do all your work
Remote	Server (GitHub, GitLab, etc.)	Shared, backed-up, team-accessible copy

Your Machine

Working Directory

Staging Area

Local Repository

GitHub / GitLab

Remote Repository
(origin/main)

push —▶
◀— fetch —

You interact with the remote primarily via `git push` (upload) and `git fetch` / `git pull` (download).

Creating a Local Repository

Start a new project from scratch

```
mkdir my-project  
cd my-project  
git init
```

```
Initialized empty Git repository in /path/to/my-project/.git/
```

What `git init` creates

```
my-project/  
└── .git/  
    ├── HEAD  
    ├── config  
    ├── objects/  
    └── refs/
```

← hidden folder – the entire repository lives here

Cloning an Existing Repository

```
# Clone via HTTPS (no SSH key required)
git clone https://github.com/org/repo.git

# Clone via SSH (requires SSH key setup)
git clone git@github.com:org/repo.git

# Clone into a specific folder name
git clone https://github.com/org/repo.git my-local-name

# Clone a specific branch
git clone --branch develop https://github.com/org/repo.git
```

What `git clone` does

1. Creates a new directory with the repository name
2. Initializes a `.git/` folder inside it
3. Downloads all history, branches, and tags from the remote
4. Sets up `origin` as the remote alias pointing to the URL you cloned from
5. Checks out the default branch (usually `main`)

After cloning, `git remote -v` shows `origin` pointing to the remote URL.

The `.gitignore` File

Tell Git which files to never track:

```
# Dependencies
node_modules/
vendor/

# Build output
dist/
*.class
*.pyc

# Environment and secrets
.env
*.pem
*.key

# OS files
.DS_Store
Thumbs.db

# IDE settings
.idea/
```

Summary — Key Commands So Far

Command	What it does
<code>git config --global user.name "..."</code>	Set your identity
<code>git init</code>	Create a new local repository
<code>git clone <url></code>	Copy a remote repository locally
<code>git config --list</code>	View all configuration
<code>.gitignore</code>	File listing patterns Git should ignore

Next module: staging changes and making your first commit.